

CSE420 Lab 2 Symbol Table Viva Cheat Sheet

CSE420 Lab 2 — Symbol Table Generation — Viva Cheat Sheet
PART 1 — THE PROBLEM, IN PLAIN ENGLISH
QUICK-FIRE FACTS FOR THE VIVA (memorize these)

CSE420 Lab 2 — Symbol Table Generation — Viva Cheat Sheet

Student ID: 22201344

PART 1 — THE PROBLEM, IN PLAIN ENGLISH

What are we building?

A compiler front-end has two jobs before it can generate code: 1. **Syntax analysis** (Lab 1) — check the source code follows the grammar of the language, using Lex (tokenizer) + Yacc (parser). 2. **Semantic bookkeeping** (Lab 2, this lab) — while parsing, also remember every identifier (variable, array, function) that gets declared: its name, its type, and **which block of code it's visible in**. This bookkeeping structure is the **Symbol Table**.

Why can't we use one flat table?

Because of **scope**. The same name can mean different things in different blocks:

```
int a, b, c;           // global a
int func(int x) {
    int t = 0;
    if (x == 1) {
        int a = 0;     // this a HIDES the global a
        t = 1;
    }
    return t;          // here, only the global a is visible again
}
```

Inside the `if` block, `a` refers to the local one. Outside it, `a` refers to the global one. A single hash table can't represent "the same name, two different meanings, depending on where you are in the code."

The solution: a stack of hash tables

- Each **block** (global scope, a function body, an if-block, etc.) gets its own **hash table**, called a **Scope Table**.
- Scope tables form a **parent-pointer chain** (like a linked stack): every scope table (except global) points back to its parent (the scope it's nested inside).
- The overall **Symbol Table** just keeps a pointer to the **current** (innermost/topmost) scope table.

Operations:

- **Enter scope** (e.g. entering `{` of a function or if-block): create a new scope table, its parent = the current one, make it current.
- **Insert** a symbol: only ever insert into the *current* (innermost) scope table.
- **Lookup** a symbol: check current scope table; if not found, check its parent; keep climbing up until found or you hit the global scope with no match.
- **Exit scope** (hitting the closing `}`): print/log the scope table (for debugging/grading), then pop back to the parent scope, and discard the finished one.

This is **exactly a call-stack-like structure**: nested scopes stack up, and unwind in LIFO order, just like function calls.

Worked example (from the spec, using their sample code)

```

1. int a,b,c;
2. int func(int x) {
3.     int t = 0;
4.     if (x == 1) {
5.         int a = 0;
6.         t = 1;
7.     }
8.     return t;
9. }
10. int main() {
11.     int x = 2;
12.     func (x);
13.     return 0;
14. }
```

At line 6 (inside the if-block), four scope tables are chained:

```

Scope3 → Scope2 → Scope1
(if)      (func)  (global)
a         x,t     a,b,c,func
```

Lookup for `a` at line 6 finds it immediately in Scope3 (the shadowing local `a`) without ever needing to check Scope1's global `a`.

How OUR code implements this (concretely)

- `symbol_info` = one declared identifier's record (name, kind, type, etc.)
- `scope_table` = one hash table (a `vector<list<symbol_info*>>`, i.e. **separate chaining**) + a pointer to its parent scope
- `symbol_table` = holds `current_scope` pointer; `enter_scope()` / `exit_scope()` push/pop; `insert()` / `lookup()` delegate to the current scope table

For **our specific test file** `input1.c`:

```

int func(int a, float b) { // Scope 1 (global): func inserted here
    return a+b;           // Scope 2: a, b (parameters) live here
}
void main () {            // Scope 1 (global): main inserted here
    int a, b, c, i;        // Scope 3: a,b,c,i,e,f,g,d all live here
    int e, f[10], g[11];
    a = 1; b = 2;
    c = func(a, b);
    float d;
}

```

- Scope 1 (global) ends up holding: `func` (Function Definition, return int, 2 params) and `main` (Function Definition, return void, 0 params).
- Scope 2 (func's body) holds: `a` (Variable, int), `b` (Variable, float) — these came from the **parameter list**, not from a `variable_decl`.
- Scope 3 (main's body) holds: `a,b,c,i` (Variable, int), `e` (Variable, int), `f` (Array, int, size 10), `g` (Array, int, size 11), `d` (Variable, float).

Notice: **function parameters share the SAME scope as the function's body** — that's a deliberate design decision our code implements (explained in Part 3).

PART 2 — THE LEX FILE (22201344.l) EXPLAINED

PART 3 — THE YACC FILE (22201344.y) EXPLAINED

What Yacc's job is Yacc (Bison) takes the stream of tokens from Lex and checks whether they form a valid sentence in our C-subset **grammar** (a set of production rules). Every time a **rule** (like `expression : logic_expression`) is fully matched, Yacc "reduces" it and runs the **action code** we attached to that rule — this is where all our symbol-table logic lives.

The 3-section structure (same idea as Lex)

Section 1 — Prologue

```

c %{ #include "symbol_table.h" #define YYSTYPE symbol_info* extern FILE *yyin; int
yyparse(void); int yylex(void); extern YYSTYPE yylval; - #include "symbol_table.h"
pulls in scope_table.h → symbol_info.h transitively, so the whole symbol-table
machinery is available. - #define YYSTYPE symbol_info* — same contract as in the .l
file: every grammar symbol (terminal or non-terminal) carries a symbol_info* as its
value. This is why every action ends with $$ = new symbol_info(...) — we're building
up a "current rendered text" of that subtree, bottom-up, purely so we can print it in
the log (it's not used for the actual symbol table logic in most rules). - yyparse ,
yylex — forward declarations of the driver and scanner-caller functions. - yylval —
same global as before, now on the Yacc side.

```

```

``c int lines = 1; ofstream outlog; symbol_table *table;

```

```

vector<pair<string,int>> decl_id_list; vector<pair<string,string>> func_param_list;
symbol_info *current_func_symbol; bool func_scope_pending = false; `` These are the
**global state variables that drive the actual symbol-table logic**: - lines — current
line number (starts at 1, incremented by Lex on every {newline} ). - outlog — the
output file stream, opened in main() . - table — the ONE symbol table for the whole
program (pointer, allocated in main() ). - decl_id_list — a scratch buffer: while
parsing a declaration_list like a, b, c, i , we push (name, arraySizeOrZero) for each
identifier here; then when the enclosing variable_decl rule reduces, we drain this
list and actually insert each one into the symbol table (because only at that point
do we know the **type** — int / float / etc — which comes from type_specifier , sitting
*before* the whole declaration_list in the rule). - func_param_list — same idea but for
function parameters: (type, name) pairs collected while parsing param_list . -
current_func_symbol — a pointer to the symbol_info object representing the function
currently being defined, so that once we finish parsing its parameter list, we can
go back and fill in param_count / param_details on the *same* object that's already
sitting inside the symbol table. - func_scope_pending — the scope-stacking
bug-fix flag (explained in detail below).

```

```

c void yyerror(char *s) { outlog<<"At line "<<lines<<" "<<s<<endl<<endl; } Yacc
calls this automatically whenever it hits a syntax error it can't recover from. We just
log it.

```

```

```c %} %token IF ELSE FOR WHILE DO BREAK INT CHAR FLOAT DOUBLE VOID
RETURN SWITCH CASE DEFAULT CONTINUE PRINTLN ADDOP MULOP INCOP DEOP
RELOP ASSIGNOP LOGICOP NOT LPAREN RPAREN LCURL RCURL LTHIRD RTHIRD
COMMA SEMICOLON CONST_INT CONST_FLOAT ID

```

%nonassoc LOWER\_THAN\_ELSE %nonassoc ELSE `` - %token ... declares every \*\*terminal symbol\*\* (token) the grammar can see — Yacc auto-assigns each one a numeric code and writes #define IF 258 etc. into y.tab.h (which Lex includes). - %nonassoc LOWER\_THAN\_ELSE / %nonassoc ELSE — this resolves the classic \*\*dangling-else ambiguity\*\* ( if(a) if(b) s1; else s2; — which if does the else belong to?). By giving ELSE higher precedence than a fake token LOWER\_THAN\_ELSE , and tagging the "IF without ELSE" rule with %prec LOWER\_THAN\_ELSE , we tell Yacc: \*prefer shifting the else` onto the nearest unmatched if\* (this is the "dangling else binds to the nearest if" convention, same as real C).

c %% End of declarations, start of grammar rules — the section you were told %% marks.

### Section 2 — Grammar rules: the general pattern Every rule looks like:

I'll now walk through the rules **in the order they matter for the symbol table** (not necessarily top-to-bottom file order), since that's what a viva examiner is most likely to grill you on.

## start : program

```

start : program
{
 outlog<<"At line no: "<<lines<<" start : program "<<endl<<endl;
 outlog<<"Symbol Table"<<endl<<endl;
 table->print_all_scopes(outlog);
}
;

```

start is the **grammar's root symbol** (declared implicitly as the first rule — Yacc always starts parsing by trying to derive start ). Once the entire program has been reduced to a single program , this action fires exactly once, at the very end, and prints the final state of the symbol table (which — since all functions have already exited their scopes — only still contains the **global scope** (Scope 1), listing main and func ).

## program and unit — just glue

```

program : program unit { ... } // multiple top-level declarations/functions, left-
recursive
 | unit { ... } // base case: one unit
 ;

unit : variable_decl { ... }
 | func_definition { ... }
 ;

```

program is **left-recursive** — this is the standard Yacc idiom for “a list of one-or-more things”: unit , or program (a smaller list) followed by one more unit . It just lets the file contain any sequence of global variable declarations and function definitions. No symbol-table logic here — purely structural.

## func\_definition — where functions get inserted, and scope gets opened

```

func_definition : type_specifier ID LPAREN
{
 func_param_list.clear();
 current_func_symbol = new symbol_info($2->get_name(),"ID");
 current_func_symbol->set_symbol_type("Function Definition");
 current_func_symbol->set_return_type($1->get_name());
 table->insert(current_func_symbol);
 table->enter_scope(outlog);
 func_scope_pending = true;
}
param_list RPAREN
{
 int p_count = 0;
 string p_details = "";
 for(unsigned int i = 0; i < func_param_list.size(); i++)
 {
 if(func_param_list[i].second != "")
 {
 symbol_info *param_sym = new
symbol_info(func_param_list[i].second,"ID");
 param_sym->set_symbol_type("Variable");
 param_sym->set_data_type(func_param_list[i].first);
 if(!table->insert(param_sym))
 {
 delete param_sym;
 }
 if(p_details != "") p_details += ", ";
 p_details += func_param_list[i].first;
 if(func_param_list[i].second != "") p_details += " " +
func_param_list[i].second;
 p_count++;
 }
 current_func_symbol->set_param_count(p_count);
 current_func_symbol->set_param_details(p_details);
 }
 compound_statement
 {
 outlog<<"At line no: "<<lines<<" func_definition : type_specifier ID LPAREN
param_list RPAREN compound_statement "<<endl<<endl;
 outlog<<$1->get_name()<<" "<<$2->get_name()<<" ("<<$5->get_name()<<")\n"<<$8-
>get_name()<<endl<<endl;
 $$ = new symbol_info($1->get_name()+" "<<$2->get_name()<<" ("<<$5-
>get_name()<<")\n"<<$8->get_name(),"func_def");
 }
 | type_specifier ID LPAREN { ...same as above... } RPAREN { p_count=0;
p_details=""; } compound_statement { ... }
;

```

**This is the rule you MUST be able to explain perfectly. Walk through it step by step:**

1. **Why is there an action right after `LPAREN`, in the middle of the rule (a “mid-rule action”)?** Because Yacc is bottom-up: it only runs a rule’s action once the *entire* right-hand side has been matched. But we need to do things (insert the function name, open a new scope) **before** the parameters and body are parsed — so we insert a mid-rule action. Bison secretly turns `A B { action } C` into a hidden empty non-terminal after `B` whose only job is to run `{ action }` — this lets us “hook in” partway through a rule.

## 2. **First mid-rule action (right after `type_specifier ID LPAREN`):**

- `func_param_list.clear()` — reset the scratch buffer for this function’s params.
- Build a `symbol_info` for the function itself: name = `$2` (the `ID` we just saw), `symbol_type` = `"Function Definition"`, `return_type` = `$1` (the `type_specifier` we just saw, e.g. `"int"`).
- `table->insert(current_func_symbol)` — **insert it into the CURRENT scope, which at this point is still the outer/global scope** (we haven’t entered a new scope yet!).

This is exactly why `func` and `main` end up living in Scope 1, not inside their own body scopes.

- `table->enter_scope(outlog)` — **now** open a brand-new scope table (this becomes Scope 2, then Scope 3, ...). This scope will hold the parameters AND the function body's local variables.
  - `func_scope_pending = true` — raise the flag telling the upcoming `compound_statement` "a scope has already been opened for you, don't open another one."
3. `param_list` — parses zero-or-more parameters, pushing `(type, name)` pairs into `func_param_list` as it goes (details below).
  4. **Second mid-rule action (right after `RPAREN`)**: now that `func_param_list` is fully populated, loop over it:
    - For each **named** parameter ( `second != ""` ), build a `Variable symbol_info` and `table->insert()` it — this inserts into the **current scope**, which is now the new function scope (Scope 2/3/...) we opened a moment ago. This is why `a` and `b` end up in Scope 2 for `func`.
    - Build up `p_details` as a human-readable string like `"int a, float b"` regardless of whether the param is named (matches the exact format in `log1.txt`: `"Parameter Details: int a, float b"` ).
    - `current_func_symbol->set_param_count(p_count) / set_param_details(p_details)` — mutate the **same object** that's already sitting inside Scope 1's hash table (we saved the pointer in `current_func_symbol` back in step 2) — so its info is correct by the time anyone prints Scope 1 later.
  5. `compound_statement` — parses the function body; internally it will just *reuse* the scope we already opened (because of the flag), insert local variable declarations into it, then close it and print it when it hits the final `}`.
  6. **Final action** — purely cosmetic logging/reconstruction of source text; no symbol-table work left to do (it already happened during the mid-rule actions and inside `compound_statement`).

**Second alternative** ( `type_specifier ID LPAREN RPAREN compound_statement` , i.e. **zero parameters**, like `void main()` ) does the exact same "insert function + open scope" mid-rule action, then since there's no `param_list` to process, the mid-rule action right after `RPAREN` just directly sets `param_count = 0` and `param_details = ""` (there was nothing to loop over).

---

`param_list` — collecting parameters into the scratch buffer

```

param_list : param_list COMMA type_specifier ID
{
 ...
 func_param_list.push_back(make_pair($3->get_name(),$4->get_name()));
}
| param_list COMMA type_specifier
{
 ...
 func_param_list.push_back(make_pair($3->get_name(),""));
}
| type_specifier ID
{
 ...
 func_param_list.push_back(make_pair($1->get_name(),$2->get_name()));
}
| type_specifier
{
 ...
 func_param_list.push_back(make_pair($1->get_name(),""));
}
;

```

Again **left-recursive** (a list built up one comma-separated item at a time). Two base cases and two “append” cases: - `type_specifier ID` — a normal named parameter (`int a`) → push `("int","a")`. - `type_specifier` alone — an **anonymous** parameter (`int` with no name, legal-ish in a prototype-only style, e.g. `int func(int, float)`) → push `("int","")`. We never insert an unnamed one into the scope (`param_count/param_details`). - The `param_list COMMA ...` versions just append one more pair to the same running list.

Notice we do NOT `.clear()` inside `param_list`’s own actions — the clearing happens once, in `func_definition`’s first mid-rule action, right before `param_list` is even entered. This matters because `param_list` recursion in Bison happens purely on the *parse stack*; nothing resets `func_param_list` mid-way through one function’s parameter list.

## variable\_decl / declaration\_list / type\_specifier — inserting ordinary variables & arrays

```

declaration_list : declaration_list COMMA ID
{
 decl_id_list.push_back(make_pair($3->get_name(),0));
}
| declaration_list COMMA ID LTHIRD CONST_INT RTHIRD
{
 decl_id_list.push_back(make_pair($3->get_name(),atoi($5->get_name().c_str())));
}
| ID
{
 decl_id_list.clear();
 decl_id_list.push_back(make_pair($1->get_name(),0));
}
| ID LTHIRD CONST_INT RTHIRD
{
 decl_id_list.clear();
 decl_id_list.push_back(make_pair($1->get_name(),atoi($3->get_name().c_str())));
}
;

```

Same left-recursive pattern as `param_list`, but for `int a, b, c, i;` - style declarations: - The **base case** (`ID` or `ID LTHIRD CONST_INT RTHIRD`) `.clear()`s the scratch list first — this is what “resets” `decl_id_list` for each new `variable_decl` statement (since every

`declaration_list` starts fresh with exactly one of these two base productions). - Array size, if present, comes from the `CONST_INT` token's `get_name()` (a string like `"10"`), converted with `atoi()` into an actual integer, stored as the pair's second element. Non-array declarations get `0` as a sentinel meaning "not an array."

```
variable_decl : type_specifier declaration_list SEMICOLON
{
 ...
 for(unsigned int i = 0; i < decl_id_list.size(); i++)
 {
 symbol_info *var_sym = new symbol_info(decl_id_list[i].first, "ID");
 if(decl_id_list[i].second > 0)
 {
 var_sym->set_symbol_type("Array");
 var_sym->set_data_type($1->get_name());
 var_sym->set_array_size(decl_id_list[i].second);
 }
 else
 {
 var_sym->set_symbol_type("Variable");
 var_sym->set_data_type($1->get_name());
 }
 if(!table->insert(var_sym))
 {
 delete var_sym;
 }
 }
}
```

Only once the **whole statement** ( `type_specifier declaration_list SEMICOLON` ) is fully parsed do we know the type ( `$1` , e.g. `"int"` ) to attach to every name collected in `decl_id_list` . We loop over the scratch buffer, build a proper `symbol_info` for each name (tagged `"Array"` if its stored size is `>0` , else `"Variable"` ), and `table->insert()` it into whatever the **current scope** happens to be at this point (global scope for top-level declarations, or the function's scope if this statement is inside a function body).

**Duplicate handling:** `table->insert()` returns `false` if the name already exists in the current scope ( `scope_table::insert_in_scope` checks via `lookup_in_scope` first). If it fails, we `delete var_sym` immediately — otherwise that heap object would leak with no owner.

`type_specifier : INT | FLOAT | VOID | CHAR` — four trivial one-token rules; each just builds a `symbol_info` whose `name` is the literal type keyword string ( `"int"` , `"float"` , etc.) — this is later read via `get_name()` wherever a "data type" or "return type" string is needed.

---

**compound\_statement — THE SCOPE-STACKING BUG FIX (very likely viva question)**

```

compound_statement : LCURL
{
 if(func_scope_pending)
 {
 func_scope_pending = false;
 }
 else
 {
 table->enter_scope(outlog);
 }
}
statements RCURL
{
 ...
 table->exit_scope(outlog);
}
| LCURL
{ <identical mid-rule action as above> }
RCURL
{
 ...
 table->exit_scope(outlog);
}
;

```

**Why this needed fixing:** `compound_statement ( { ... } )` is used in **two different situations** in this grammar: 1. As a **function body** — immediately following `func_definition`'s parameter list, where a scope was **already opened** by `func_definition`'s mid-rule action (so params and body share one scope). 2. As the body of an `if`, `while`, or `for` **nested inside** a function (via `statement : compound_statement`) — where **no scope has been opened yet** for this block; it needs to open (and later close) its own.

The bug (before the fix): `exit_scope()` was called unconditionally on every `RCURL`, but `enter_scope()` was only ever called by `func_definition`. So the very first `if(...) { ... }` inside any function would hit its closing `}`, call `exit_scope()`, and **pop the function's own scope right out from under it** — corrupting everything parsed afterward (variables declared later in the function would wrongly land in the *global* scope, or worse).

**The fix — a global flag, `func_scope_pending`:** - `func_definition` sets it to `true` immediately after opening the function's scope. - The very next thing parsed is guaranteed to be that function's `compound_statement` (nothing else can come between a function's `)` and its `{}`, so `compound_statement`'s new mid-rule action (right after `LCURL`) checks the flag: - **If `true`** (this brace is the function's own body): consume the flag (`= false`) and do nothing — the scope is already open and already has the parameters in it. - **If `false`** (this brace is a nested block inside a function, e.g. inside `if`): call `table->enter_scope(outlog)` ourselves, right now. - Either way, `table->exit_scope(outlog)` is now called unconditionally and *correctly balances* whichever `enter_scope()` call actually happened for this particular pair of braces.

**Why is this specific location ( `right after LCURL` , before `statements` ) chosen, and not simply inside the final action after `RCURL` ?** Because we need to decide/act *before* any statements inside the block get parsed (those statements' `variable_decl` actions need `table->insert()` to already be inserting into the *correct*, freshly-opened scope) — this is another mid-rule action, for the same underlying reason as in `func_definition`.

**Test with `input1.c` (no nested if/while/for):** `func_scope_pending` is `true` at both compound-statements (func's and main's), so behavior is identical to “no flag at all” — matching `log1.txt` exactly. The fix only changes behavior for grammars with nested blocks, which aren't exercised by this particular input, but the fix is what makes the code *correct in general*, not just correct-by-luck for this one test file.

---

## The rest of the grammar (expression/statement machinery) — quick tour

These rules exist purely to parse **expressions and statements syntactically** (this was already done correctly in Lab 1) — **none of them insert anything into the symbol table**; they just rebuild the matched source text bottom-up into `$$` so the log can print it. If asked about any single one of these, the safe generic answer is:

“This rule doesn't touch the symbol table — it's inherited from Lab 1's syntax analyzer. Its job is purely to recognize this piece of grammar and reconstruct/print the matched text; `$$` here is only used for logging, built by concatenating children's text with `get_name()`.”

- `statements : statement | statements statement` — a list of statements (again left-recursive).
  - `statement : variable_decl | func_definition | expression_statement | compound_statement | FOR(...) | IF(...) | IF(...)ELSE(...) | WHILE(...) | PRINTLN(...) | RETURN expr;` — every kind of statement our C-subset supports.
  - `expression_statement : SEMICOLON | expression SEMICOLON` — an expression followed by `;`, or just a bare `;`.
  - `variable : ID | ID LTHIRD expression RTHIRD` — a plain name or an array-indexed access (`a` or `f[i]`).
  - `expression : logic_expression | variable ASSIGNOP logic_expression` — either a value, or an assignment.
  - `logic_expression → rel_expression → simple_expression → term → unary_expression → factor_info → factor` — this whole chain implements **standard operator precedence** via grammar layering (the classic Yacc technique — each precedence level is its own non-terminal, lowest precedence at the top): logical (`&&`, `||`) < relational (`<`, `>`, `==`, `!=`) < additive (`+`, `-`) < multiplicative (`*`, `/`, `%`) < unary (`!`, unary `+`/`-`) < atomic factor.
  - `factor : variable | ID LPAREN argument_list RPAREN | LPAREN expression RPAREN | CONST_INT | CONST_FLOAT | variable INCOP | variable DECOP` — the “atoms”: a variable, a function call, a parenthesized expression, a literal, or post-increment/decrement.
  - `argument_list : arguments | <empty>` and `arguments : arguments COMMA logic_expression | logic_expression` — the comma-separated list inside a function call's `(...)`.
- 

## Section 3 — `main()`

```

int main(int argc, char *argv[])
{
 if(argc != 2)
 {
 cout<<"Please input file name"<<endl;
 return 0;
 }
 yyin = fopen(argv[1], "r");
 outlog.open("22201344_log.txt", ios::trunc);

 if(yyin == NULL)
 {
 cout<<"Couldn't open file"<<endl;
 return 0;
 }
 table = new symbol_table(10);
 outlog<<"New ScopeTable with ID 1 created"<<endl<<endl;

 yyparse();

 outlog<<endl<<"Total lines: "<<lines<<endl;

 outlog.close();
 fclose(yyin);
 delete table;
 return 0;
}

```

Line by line: - `argc != 2` — expects exactly one command-line argument (the input filename) besides the program name itself; else prints usage and exits. - `yyin = fopen(argv[1], "r")` — `yyin` is Flex's global "which file to read tokens from"; by default it's `stdin`, so we redirect it to our input `.c` file. - `outlog.open("22201344_log.txt", ios::trunc)` — opens (and truncates/overwrites) the log file — **this exact filename is required by the submission spec** (`<student_id>_log.txt`). - `table = new symbol_table(10)` — allocates the ONE symbol table with **10 buckets per scope** (bucket count chosen/verified to reproduce the exact hash-bucket numbers seen in the sample `log1.txt`). - The constructor of `symbol_table` (in `symbol_table.h`) **already creates Scope 1 internally** — but its constructor has no access to `outlog`, so we manually print the matching `"New ScopeTable with ID 1 created"` log line right here, immediately after. - `yyparse()` — hands control to the generated parser, which repeatedly calls `yylex()` (Lex) to pull tokens and drives the whole grammar/actions described above. This is where basically all the real work happens. - After parsing finishes, print the total line count (per spec requirement #4), close the file, close the input file, and `delete table` (which cascades — `~symbol_table()` walks up the parent chain deleting every remaining scope table, whose own destructor deletes every `symbol_info*` still stored in it — full cleanup, no leaks for symbols still in the table when the program ends).

## ## PART 4 — THE THREE HEADER FILES

### ### 4.1 `symbol_info.h` — one declared identifier's record

```
``cpp #include<bits/stdc++.h> using namespace std;
```

```
class symbol_info { private: string name; // the identifier's literal text, e.g. "a", "func"
string type; // the LEXICAL token type from Lex, e.g. "ID", "ADDOP", "INT" — NOT the C
data type! string symbol_type; // "Variable" | "Array" | "Function Definition" — what
KIND of symbol this is string data_type; // the C data type: "int", "float", "char", "void"
(for variables/arrays) string return_type; // the C return type (for functions only) int
param_count; // number of parameters (functions only) string param_details; //
human-readable "int a, float b" (functions only) int array_size; // size (functions/vars:
0; arrays: the declared size) ```
```

**Common viva trap:** `type` vs `data_type` vs `symbol_type`. **Know the difference**  
**cold:** - `type` — set once, at **construction time**, by whoever created this object

(usually Lex, sometimes the parser). For an `ID` token it's literally the string `"ID"`. It's really more of a "constructor tag" than semantic info. - `symbol_type` — set **later**, by the parser, once it knows *what kind* of thing this identifier is: `"Variable"`, `"Array"`, or `"Function Definition"`. - `data_type` — the actual C type (`"int"`, `"float"`, ...) — set by the parser once the `type_specifier` for this declaration is known.

```
cpp symbol_info(string name, string type) { this->name = name; this->type = type;
this->symbol_type = ""; this->data_type = ""; this->return_type = ""; this-
>param_count = 0; this->param_details = ""; this->array_size = 0; } Constructor
takes only name and type — everything else defaults to empty/zero and gets filled in
later via setters, because at construction time (e.g., the moment Lex sees an ID) we
don't yet know if it'll turn out to be a variable, array, or function.
```

Then it's just a plain **getter/setter pair for every field** ( `get_name/set_name`, `get_type/set_type`, `get_symbol_type/set_symbol_type`, `get_data_type/set_data_type`, `get_return_type/set_return_type`, `get_param_count/set_param_count`, `get_param_details/set_param_details`, `get_array_size/set_array_size` ). Nothing clever — it's a data class ("plain old data" with encapsulation).

```
cpp ~symbol_info() { } Empty destructor — symbol_info doesn't own any heap
memory of its own (just string/int members which clean up themselves), so
there's nothing manual to free here. (Compare: scope_table's destructor DOES need
to manually delete things, because it stores raw pointers.)
```

## 4.2 `scope_table.h` — one hash table representing one scope/block

```
class scope_table
{
private:
 int bucket_count; // how many buckets/lists in this hash table
 (we use 10)
 int unique_id; // this scope's ID number (1,2,3,...) for
 logging
 scope_table *parent_scope = NULL; // pointer up the chain to the enclosing scope
 vector<list<symbol_info *>> table; // the hash table itself: array of buckets,
 each a linked list
```

This is **separate chaining**: `table` is a `vector` of size `bucket_count`; each slot ( `table[i]` ) is a `list<symbol_info*>` holding every symbol whose hash lands on bucket `i` — collisions just get appended to that bucket's list rather than causing a clash.

```
int hash_function(string name)
{
 int sum = 0;
 for (int i = 0; i < name.length(); i++)
 sum = sum + (int)name[i];
 return sum % bucket_count;
}
```

The hash function: sum up the ASCII value of every character in the name, then mod by the bucket count. Simple but sufficient for this lab.

**Example you should be able to do live:** `"a"` → ASCII 97 →  $97 \% 10 = 7$ . `"main"` →  $109+97+105+110 = 421 \rightarrow 421 \% 10 = 1$ . `"func"` →  $102+117+110+99 = 428 \rightarrow 428 \% 10 = 8$ . These are exactly the bucket numbers you see in the sample log — memorize this trick, an examiner may ask you to compute one by hand.

```
scope_table::scope_table(int bucket_count, int unique_id, scope_table *parent_scope)
{
 this->bucket_count = bucket_count;
 this->unique_id = unique_id;
 this->parent_scope = parent_scope;
 table.resize(bucket_count); // allocate 'bucket_count' empty buckets
}
```

The “real” constructor, used every time a new scope is entered — note it takes the **parent** scope as a parameter, which is how the chain gets wired up (this is what lets `lookup` climb outward later).

```
symbol_info *scope_table::lookup_in_scope(symbol_info* symbol)
{
 int index = hash_function(symbol->get_name());
 for (list<symbol_info *>::iterator it = table[index].begin(); it !=
table[index].end(); it++)
 if ((*it)->get_name() == symbol->get_name())
 return *it;
 return NULL;
}
```

Hash to the right bucket, then linearly scan that bucket’s list comparing names — **only searches THIS scope**, never the parent (that’s `symbol_table::lookup`’s job, one level up).

```
bool scope_table::insert_in_scope(symbol_info* symbol)
{
 if (lookup_in_scope(symbol) != NULL) return false; // no duplicates allowed in the
same scope
 int index = hash_function(symbol->get_name());
 table[index].push_back(symbol);
 return true;
}
```

Insert only if not already present in *this* scope (checked via a lookup first) — this is exactly why our `.y` code has to check `if(!table->insert(...)) delete ...;` — a `false` return means “already declared, I own nothing new to store, so free what I just allocated.”

```
void scope_table::print_scope_table(ofstream& outlog)
{
 outlog << "ScopeTable # " + to_string(unique_id) << endl;
 for (int i = 0; i < bucket_count; i++)
 {
 if (!table[i].empty())
 {
 outlog << i << " -> " << endl;
 for (...)
 {
 symbol_info *s = *it;
 outlog << "< " << s->get_name() << " : " << s->get_type() << " >" << endl;
 if (s->get_symbol_type() == "Variable") { ...print Type... }
 else if (s->get_symbol_type() == "Array") { ...print Type, Size... }
 else if (s->get_symbol_type() == "Function Definition") { ...print Return
Type, Param Count, Param Details... }
 }
 outlog << endl;
 }
 }
}
```

Prints buckets **in index order (0..bucket\_count-1)**, skipping empty ones — this is exactly the format you see reproduced in `log1.txt` (`0 ->`, `1 ->`, etc., only for non-empty buckets). For each symbol it prints `< name : lexical_type >` then a kind-specific detail block depending on `symbol_type`.

```
scope_table::~~scope_table()
{
 for (int i = 0; i < bucket_count; i++)
 {
 for (...) delete (*it); // free every symbol_info* stored here
 table[i].clear();
 }
}
```

Since a scope table stores raw pointers ( `symbol_info*` ), the compiler will NOT automatically free the pointed-to objects when a `list<symbol_info*>` is destroyed (it only destroys the pointers themselves, which does nothing to the heap memory). So we manually `delete` every stored pointer here — this runs whenever a scope is popped ( `symbol_table::exit_scope` deletes the `scope_table` object, triggering this destructor) or when the whole program ends ( `symbol_table` 's destructor walks the chain deleting each one).

---

#### 4.3 `symbol_table.h` — the whole table, i.e. the “stack of scope tables”

```
class symbol_table
{
private:
 scope_table *current_scope; // pointer to the innermost/topmost scope right now
 int bucket_count; // remembered so every new scope uses the same size
 int current_scope_id; // running counter — next scope created gets id = this+1
```

```
symbol_table::symbol_table(int bucket_count)
{
 this->bucket_count = bucket_count;
 this->current_scope_id = 1;
 current_scope = new scope_table(bucket_count, current_scope_id, NULL);
}
```

Constructor immediately creates **Scope 1** (the global scope) with `parent_scope = NULL` (nothing above global). This is why `main()` in the `.y` file doesn't need to call `enter_scope()` for the global scope — it already exists the moment `new symbol_table(10)` runs.

```
symbol_table::~symbol_table()
{
 while (current_scope != NULL)
 {
 scope_table *temp = current_scope;
 current_scope = current_scope->get_parent_scope();
 delete temp;
 }
}
```

Walks the entire remaining chain (should normally just be Scope 1 by the time the program ends, since every function properly popped its own scope on the way) and deletes each one — cascading into `scope_table` 's destructor, which frees every remaining `symbol_info*`.

```
void symbol_table::enter_scope(ofstream& outlog)
{
 current_scope_id++;
 scope_table *new_scope = new scope_table(bucket_count, current_scope_id,
current_scope);
 current_scope = new_scope;
 outlog << "New ScopeTable with ID " << current_scope_id << " created" << endl << endl;
}
```

**This is the fix we made.** Bump the ID counter, create a brand-new scope table whose parent is **whatever was current a moment ago** (this is the actual “push” — wiring the new scope onto the top of the stack), make it current, and log the creation. Taking `ofstream& outlog` as a parameter (rather than logging externally in the `.y` file) keeps the logging responsibility co-located with the state change that causes it.

```

void symbol_table::exit_scope(ofstream& outlog)
{
 print_all_scopes(outlog);
 int removed_id = current_scope->get_unique_id();
 scope_table *temp = current_scope;
 current_scope = current_scope->get_parent_scope();
 delete temp;
 outlog << "Scopetable with ID " << removed_id << " removed" << endl << endl;
}

```

**This is the other half of the fix.** Order matters here: 1.

`print_all_scopes(outlog)` FIRST — while the scope being exited is still `current_scope`, print the whole chain (spec requirement: “when you exit a scope print the current state of the symbol table”). 2. THEN capture its id, “pop” ( `current_scope = parent` ), `delete` the old scope object (triggers `~scope_table()`, freeing its symbols). 3. THEN log the removal message, using the **captured** id (we can’t ask `current_scope->get_unique_id()` for this anymore — `current_scope` now points at the parent!).

```

bool symbol_table::insert(symbol_info* symbol) { return current_scope->insert_in_scope(symbol); }

```

Pure delegation — always inserts into whatever is currently on “top of the stack.” This is the single line that makes the whole “insert always goes into the innermost scope” rule work.

```

symbol_info* symbol_table::lookup(symbol_info* symbol)
{
 scope_table *temp = current_scope;
 while (temp != NULL)
 {
 symbol_info *found = temp->lookup_in_scope(symbol);
 if (found != NULL) return found;
 temp = temp->get_parent_scope();
 }
 return NULL;
}

```

**This is the “climb the parent chain” logic** described in Part 1: start at the current (innermost) scope, check it; if not found, move to its parent; repeat until found or you fall off the end (global scope’s parent is `NULL`). This naturally implements shadowing: the innermost match wins, because we stop as soon as we find one.

```

void symbol_table::print_all_scopes(ofstream& outlog)
{
 outlog<<"#####"<<endl<<endl;
 scope_table *temp = current_scope;
 while (temp != NULL)
 {
 temp->print_scope_table(outlog);
 temp = temp->get_parent_scope();
 }
 outlog<<"#####"<<endl<<endl;
}

```

Prints **every** scope currently on the stack, from innermost to outermost, sandwiched between `####` banners — this exactly matches the blocks you see in `log1.txt` (e.g. when Scope 2 exits, you see `ScopeTable # 2` printed first, then `ScopeTable # 1` right after it, both between one pair of banners).

<pre> c 1.  int func(int a, float b) { 2.      return a+b; 3.  } 4.  void main () { 6. int e, f[10], g[11]; 8.      a = 1; 9.      b = 2; 10.      c = func(a, b); 11. 12. </pre>
### Step 0 — Program start <code>main()</code> in <code>22201344.y</code> runs: opens <code>22201344_log.txt</code>, crea <code>symbol_table(10)</code>. The <code>symbol_table</code> constructor silently creates <b>Scope 1</b> internally. We
### Step 1 — Parsing <code>int func(int a, float b) {</code> Lex tokenizes: <code>INT</code> <code>ID("func")</code> <code>LPAR</code> <code>FLOAT</code> <code>ID("b")</code> <code>RPAREN</code> <code>LCURL</code>.
Yacc reduces <code>type_specifier : INT</code> → prints <code>int</code>. Then sees <code>ID</code>, then <code>LPAREN</code> → <b>fires 1</b>  <b>mid-rule action:</b> - Creates <code>symbol_info("func","ID")</code>, sets <code>symbol_type="Function Defin</code>  <code>return_type="int"</code>. - <code>table-&gt;insert(current_func_symbol)</code> → goes into <b>Scope 1</b> (still cur opened a new scope yet). - <code>table-&gt;enter_scope(outlog)</code> → creates <b>Scope 2</b>, parent = S
Then <code>param_list</code> is parsed: - <code>type_specifier : INT</code> → <code>"int"</code>. Reduce <code>param_list : type</code>  → pushes <code>("int","a")</code> into <code>func_param_list</code>, prints <code>int a</code>. - <code>type_specifier : FLOAT</code> →  <code>param_list : param_list COMMA type_specifier ID</code> for the whole <code>int a, float b</code> → pushe  <code>int a,float b</code>.
<code>RPAREN</code> triggers the <b>second mid-rule action</b> of <code>func_definition</code>: - Loop over <code>func_par</code>  <code>("float","b")]</code> : - <code>("int","a")</code> : named → build <code>symbol_info("a","ID")</code>, <code>symbol_type="Var</code>  <code>table-&gt;insert()</code> → goes into <b>Scope 2</b> (current scope right now). <code>p_details = "int a"</code>. -  <code>symbol_info("b","ID")</code>, <code>Variable</code>, <code>data_type="float"</code>, inserted into <b>Scope 2</b>. <code>p_details</code>  <code>current_func_symbol-&gt;set_param_count(2)</code>, <code>set_param_details("int a, float b")</code> — this n  that's sitting inside <b>Scope 1</b>.
### Step 2 — Parsing <code>{ return a+b; }</code> (the function body, line 1-3) <code>LCURL</code> fires <code>compou</code>  action: <code>func_scope_pending</code> is <code>true</code> → <b>consume it</b> (<code>= false</code>), do NOT open a new scop  2, which already has <code>a</code> and <code>b</code> in it).
<code>return a+b;</code> is parsed through the expression grammar chain ( <code>variable</code>→<code>factor</code>→...→<code>exp</code>  string <code>"a+b"</code>, eventually reducing <code>statement : RETURN expression SEMICOLON</code> → <code>"return a</code>  touch the symbol table.
<code>RCURL</code> fires the final action: - Prints the reconstructed compound statement text. - <code>tabl</code>  called: 1. <code>print_all_scopes(outlog)</code> — prints, from innermost to outermost: ``  #####
<pre> ScopeTable # 2 7 -&gt; &lt; a : ID &gt; Variable Type: int 8 -&gt; &lt; b : ID &gt; Variable Type: float </pre>
<pre> ScopeTable # 1 8 -&gt; &lt; func : ID &gt; Function Definition Return Type: int Number of Param Details: int a, float b ##### </pre>
<code>func_definition</code>'s own final action then prints the whole reconstructed function text an  unit, program.
### Step 3 — Parsing <code>void main () {</code> Same dance as Step 1: - <code>type_specifier : VOID</code>  <code>"main"</code>, <code>LPAREN</code> → mid-rule action: <code>symbol_info("main","ID")</code>, <code>Function Definition</code>, <code>ret</code>  <code>&gt;insert()</code> → <b>Scope 1</b> (current is back to Scope 1, since Scope 2 was already popped).  <code>&gt;enter_scope(outlog)</code> → creates <b>Scope 3</b>, parent = Scope 1, prints:
### Step 4 — Parsing the body of <code>main</code>, lines 6-12 <code>LCURL</code> → <code>func_scope_pending</code> is <code>true</code>  Scope 3.
Then, statement by statement:
<b>Line 6:</b> <code>int a, b, c, i;</code> - <code>type_specifier : INT</code> → <code>"int"</code>. - <code>declaration_list : ID</code> for  <code>decl_id_list.clear()</code>, push <code>("a",0)</code>. - <code>declaration_list : declaration_list COMMA ID</code> fo  pushes <code>("b",0)</code>, <code>("c",0)</code>, <code>("i",0)</code>. Now <code>decl_id_list = [(a,0),(b,0),(c,0),(i,0)]</code>. - <code>v</code>  <code>type_specifier declaration_list SEMICOLON</code> reduces → loop over <code>decl_id_list</code>, for each l  <code>data_type="int"</code>, <code>table-&gt;insert()</code> into <b>Scope 3</b>. All four go in.
<b>Line 7:</b> <code>int e, f[10], g[11];</code> - <code>type_specifier : INT</code> → <code>"int"</code>. - <code>declaration_list : ID</code>  (wipes the previous statement's leftover list!) then push <code>("e",0)</code>. - <code>declaration_list :</code>  <code>ID LTHIRD CONST_INT RTHIRD</code> for <code>f[10]</code> → push <code>("f", atoi("10")) = ("f",10)</code>. - Same ag  <code>("g",11)</code>. - <code>variable_decl</code> reduces → loop: <code>e</code> → <code>Variable,int</code>; <code>f</code> → <code>Array,int,size=10</code> (si  <code>g</code> → <code>Array,int,size=11</code>. All inserted into <b>Scope 3</b>.
<b>Lines 8-9:</b> <code>a = 1; b = 2;</code> — pure expression statements ( <code>variable ASSIGNOP logic_expr</code>  insertions — <code>a</code> and <code>b</code> here are just <i>uses</i>, not declarations (the grammar doesn't distin  insert on <code>variable_decl</code> /parameter/function rules).
<b>Line 10:</b> <code>c = func(a, b);</code> - Parsed as <code>factor : ID LPAREN argument_list RPAREN</code> for the c  inside <code>expression : variable ASSIGNOP logic_expression</code>. Purely syntactic — no symbol t  doesn't implement type-checking/argument-count validation against the stored functio  syntax analyzer prints "type mismatch" lines from its OWN separate hard-coded logic, i  table additions — those lines you saw in <code>log1.txt</code> come from Lab 1 code, not somethin
<b>Line 12:</b> <code>float d;</code> - <code>type_specifier : FLOAT</code> → <code>"float"</code>. - <code>declaration_list : ID</code> for <code>d</code>

```

("d",0) . - variable_decl reduces → d → Variable, float, inserted into Scope 3.

Step 5 — Closing main's body, line 13 } RCURL for the outer compound_statement
>exit_scope(outlog) : 1. print_all_scopes — prints Scope 3 (innermost) then Scope 1: ``
#####

ScopeTable # 3 0 -> < d : ID > Variable Type: float 1 -> < e : ID > Variable Type: int 2 -
int Size: 10 3 -> < g : ID > Array Type: int Size: 11 5 -> < i : ID > Variable Type: int 7 -:
Type: int 8 -> < b : ID > Variable Type: int 9 -> < c : ID > Variable Type: int

ScopeTable # 1 1 -> < main : ID > Function Definition Return Type: void, Params: 0 8 -:
Definition Return Type: int, Params: 2, "int a, float b"

`` (Check the bucket math
yourself: d =100%10=0, e =101%10=1, f =102%10=2, g =103%10=3, i =105%10=5, a =97%10=7, b =91
(109+97+105+110)=421%10=1, func =428%10=8.) 2. Pops Scope 3, deletes it (frees d,e,f,g,i,
Prints Scopetable with ID 3 removed`.

Step 6 — End of file, start : program fires The whole file has now reduced up to a
start : program's action runs: `` Symbol Table

#####

ScopeTable # 1 1 -> < main : ... > 8 -> < func : ... >

`` Only Scope 1 remains (Scope:
popped along the way) — this is the program's final, permanent symbol table: the two t
declarations.

Step 7 — Cleanup Back in main() (the C++ main, not the parsed C main function
13, close the log file, close the input file, delete table → destructor walks the (now sin
deletes Scope 1, whose destructor frees main and func's symbol_info objects. No leak

```

## QUICK-FIRE FACTS FOR THE VIVA (memorize these)

- **Bucket count = 10.** Chosen because it reproduces the exact bucket indices in the sample log.
- **Hash function:** sum of ASCII codes of every character in the name, mod bucket count.
- **3 scopes total for input1.c :** Scope 1 = global ( func , main ), Scope 2 = func 's body+params ( a , b ), Scope 3 = main 's body+params ( d,e,f,g,i,a,b,c — no params since main has none).
- **Parameters live in the SAME scope as the function body** (not a separate scope) — this is a deliberate design choice, implemented by opening the scope right after LPAREN (before parsing param\_list ), and having compound\_statement reuse it via the func\_scope\_pending flag instead of opening a second one.
- **Function symbols are inserted into the PARENT scope, before the new scope is opened** — that's why func and main end up in Scope 1, not their own body scopes.
- **func\_scope\_pending flag** exists to fix a bug where nested if/while/for blocks would incorrectly pop the function's own scope when hitting their closing } (since compound\_statement unconditionally called exit\_scope , but only func\_definition used to call enter\_scope ).
- **Duplicate declarations are silently rejected:** scope\_table::insert\_in\_scope returns false if the name is already present in that scope; the .y code then delete s the unused symbol\_info to avoid a memory leak.
- **\$\$, \$1, \$2, ...** are Yacc's way of referring to the value of the rule being built ( \$\$ ) and its right-hand-side symbols in order ( \$1 = first symbol, etc.). Every symbol's value type is symbol\_info\* (set via

```
#define YYSTYPE symbol_info*).
```

- `%%` appears twice in both `.l` and `.y` files: the first separates *definitions* from *rules*; the second separates *rules* from *extra user C code* (only used in the `.y` file here, for `main()` ).
- `yylval` is the bridge variable Lex sets before returning a token, which Yacc automatically reads into `$n` for that token's position.
- **Log filename:** `22201344_log.txt` , per the `<student_id>_log.txt` submission requirement.
- **Mid-rule actions** are how we run code *before* a Yacc rule is fully matched (e.g., opening a scope before parsing parameters) — Bison implements them as a hidden zero-length nonterminal inserted at that point in the rule.